

ETOGO Property Twin - Response for Engineering Discussion

Prepared for the property twin engineer | July 27, 2026

Important hosting constraint: the application is currently hosted on Laravel Cloud. We should assume the Laravel Cloud runtime will not run heavy native conversion tools such as PDAL, PotreeConverter, IFC converters, or large point-cloud processing directly inside the web app. Laravel Cloud should orchestrate the workflow; heavy conversion should run in a separate worker service.

Direct Answers

CLOUD AND DATA RESIDENCY

We have not finalized AWS, Azure, or GCP as the long-term storage/processing provider. Because the main app is currently on Laravel Cloud, the near-term architecture should remain cloud-agnostic through Laravel storage adapters.

Recommended baseline: private object storage, encryption at rest and in transit, immutable original source files, checksums, signed/temporary access URLs, and audit logs for all upload, conversion, linking, marker movement, verification, and stewardship events. Final production region should be selected after client/data-residency requirements are confirmed.

VIEWER STACK AND OFFLINE MOBILE

We prefer Three.js/WebGL for the first production version because the current Laravel/Vite application already uses Three.js for GLB/gITF models, panoramas, and marker placement. WebGPU can be considered later, but WebGL is the safer near-term path.

Offline mobile viewing is not required for the MVP. The model should not block future read-only/offline mobile support, but the current priority is reliable web viewing, governed evidence ingestion, conversion status, and fast progressive loading.

Current Application Stack

Area	Current Structure
Backend	PHP 8.2 with Laravel 12
Frontend	Blade templates, Bootstrap-style admin/client UI, Vite
Viewer layer	JavaScript with Three.js/WebGL

Database	Current Laravel relational schema using migrations
Storage	Laravel filesystem abstraction; currently local/public, should move toward private object storage
Background jobs	Laravel queues
Current conversion path	PDAL plus PotreeConverter exists in code, but should run outside Laravel Cloud in a dedicated worker/container

Current Twin Entities and Flow

The current conceptual flow is:

Property -> Inspection/Diagnosis -> Capture Session -> Spatial Model -> Issue Marker -> PHAR Finding -> Evidence -> Quotation -> Work Order -> Verification -> Verified Fact -> Stewardship

Relevant entities already exist or are being introduced in the application:

- properties
- inspections as the operational diagnosis/assessment record
- capture_sessions
- spatial_models
- matterport_models for legacy Matterport support
- issue_markers
- phar_findings
- finding_evidence
- finding_client_decisions
- remediation_roadmaps
- remediation_work_orders
- verification_records
- verified_property_facts
- stewardship_plans

Recommended Cloud Architecture

Because Laravel Cloud should not be treated as the conversion environment, the preferred architecture is:

Laravel Cloud app -> private object storage -> external conversion worker -> converted tiles/models -> Laravel viewer

Laravel Cloud Responsibilities

- User authentication and authorization
- Property, inspection, capture, twin, finding, evidence, verification, and stewardship records
- Upload orchestration and metadata capture

- Checksum and immutability rules
- Conversion job status: queued, processing, ready, failed
- Viewer pages and signed/private access URLs
- Audit trail and governed workflow events

External Conversion Worker Responsibilities

- Pull original source files from object storage
- Convert E57, LAS, LAZ, IFC, OBJ/mesh, or other heavy formats into browser-ready outputs
- Write converted outputs back to object storage
- Call Laravel back through an API or webhook
- Update conversion status and error details

Position on Proposed Architecture

The proposed graph/time/spatial direction is strong, especially for governed evidence and large buildings. We agree with the principles: immutable sources, spatial tiling, LOD, lazy evidence loading, and reviewed anchor events.

However, we recommend a phased approach. Do not introduce Neo4j, PostGIS, and Timescale all at once unless the immediate query and scale requirements justify the infrastructure. First stabilize the ontology, IDs, immutability model, event history, and conversion contract inside the current Laravel architecture. Then add PostGIS, graph, or time-series layers when the use cases require them.

Viewer and Format Strategy

- Use Three.js/WebGL first.
- Use GLB/glTF for browser-ready runtime models.
- Use Potree or 3D Tiles for large point clouds and high-rise scale.
- Use spatial tiling, floor/zone segmentation, and LOD.
- Lazy-load evidence, markers, floors, and heavy layers.
- Consider WebGPU later after the WebGL version is stable.

Recommended First Deliverable

1. Define the Property Twin ontology and stable IDs.
2. Define the evidence registry and immutable source-file rules.
3. Define checksum, metadata, and storage path conventions for originals and converted outputs.
4. Define conversion job lifecycle and the external worker API/webhook contract.
5. Define ingestion/conversion handling for Matterport, IFC, E57, LAS/LAZ, GLB/glTF, images, PDFs, panoramas, and mesh packages.
6. Define anchor governance rules: preserve, relocate, split, merge, retire, and supersede, all with reviewed events.
7. Define the viewer loading strategy for large properties, including 56-floor high-rise scenarios.

Summary

We are Laravel Cloud plus Laravel/PHP and Three.js today. Laravel Cloud should remain the application and workflow layer. Heavy IFC/E57/point-cloud conversion should be handled by a separate processing service connected to private object storage. The long-term goal is a governed, scalable property twin with immutable evidence, stable IDs, reviewed anchor events, progressive loading, and a clean path toward graph/time/spatial storage when needed.

Internal note: this document reflects the current ETOGO architecture and the recommended direction for the external property twin engineering work.